CECS 327 Distributed Lodging — Milestone 2: Communication Mechanisms Report

Course: CECS 327, Section 02, CSULB **Repository:** https://github.com/haileyc1201/CECS327-Distributed-Lodging **Branches:** `main` (Python simulation) · `tom-react-frontend` (live Node + React + Cloudflare Workers/D1 version)

1. Overview: one protocol, two paths

The system has two implementations that use the **same message envelope and wire formats**:

- **Path A: Python simulation (`main`).** Each node is its own OS process: Property RPC, Reservation RPC, a pub-sub Broker, a Group Coordinator, 3 subscriber services and 3 property replicas. They talk over **raw TCP sockets** with length-prefixed frames carrying **JSON-RPC 2.0** and event messages. It builds on Hailey's Milestone 1 Property/Reservation socket services and the same `data/properties.json`. Her original services and client still run unchanged.
- **Path B: live web version (`tom-react-frontend`).** There are two runtimes:
- **Local Node stack.** Express microservices (Property 5001, Reservation 5002, Payment 5003, Gateway 8000), plus a new **Comm Hub** (5010) and 6 comm nodes (3 subscribers, 3 replicas) as separate processes, communicating with **HTTP JSON-RPC** and HTTP push.
- **Hosted Cloudflare Worker.** A Hono Worker with D1 that exposes **HTTP JSON-RPC** at `/api/rpc`. A **Durable Object (`CommHub`)** acts as broker and group coordinator. External subscribers and members connect over **WebSockets**. Every message is logged to a D1 table served at `/api/comm/logs`.

Because the envelope and the XML mapping are identical, a message marshalled by the Python codec can be unmarshalled by the Node/TypeScript codec. This was checked during development.

2. Communication model

Aspect	Path A: Python simulation (`main`)	Path B: Node local / Cloudflare hosted (`tom-react-frontend`)
Transport	TCP sockets (`socket` module), 4-byte length + 1-byte format header per frame	HTTP/1.1 (Node `fetch` + Express). Cloudflare: HTTPS to Worker, Worker→Durable Object stub fetch, WebSockets for push
Processes / nodes	10 OS processes: property-rpc :6001, reservation-rpc :6002, broker :6010, group-coordinator :6020, 3 subscribers, 3 replicas :6101-6103	Local: 11 processes (4 original services + hub + 3 subscribers + 3 replicas). Hosted: Worker isolates (stateless, global edge) + one `CommHub` Durable Object (single, strongly consistent)
Synchronous style	Blocking request-reply on a persistent socket	HTTP request-reply. The Worker awaits the DO reply
Asynchronous style	Broker pushes `event` frames over long-lived subscriber sockets	Local: hub POSTs `event` to each subscriber's `/deliver`. Hosted: DO pushes over WebSocket, built-in subscribers run inside the DO, mailbox members poll `group.inbox`
Shared state	In-memory per node; property data from `data/properties.json` (not modified by demo)	Local: CSV files in `data/`. Hosted: D1 (`properties`, `reservations`, `message_log`) + DO storage (membership, inboxes, sequence numbers)

Booking flow (both paths): **client → Reservation (RPC) → Property `availability.check` (request-reply) → mark unavailable → publish `booking.created` (pub-sub) → multicast `availability_update` to group `property-replicas` (group communication) → reply to client** listing which subscribers and replicas were reached.

3. Message formats

Every message, on every path, is an **envelope**:

```
{
  "id": "fc94cb9cacac",
  "type": "rpc.request",
  "sender": "client",
  "receiver": null,
  "timestamp": "2026-10-12T23:16:20.185Z",
  "correlation_id": null,
  "payload": {"jsonrpc": "2.0", "method": "reservation.book",
              "params": {"property_id": 101, "guest_name": "Demo Guest", "nights": 2}, "id": 3}
}
```

The reply carries `correlation_id` = the request's `id`, which lets clients match replies to requests:

```
{"id":"a13de2f6078d","type":"rpc.reply","sender":"reservation-rpc","receiver":"client",
 "timestamp":"2026-10-12T23:16:20.195Z","correlation_id":"fc94cb9cacac",
 "payload":{"jsonrpc":"2.0","id":3,"result":{"status":"accepted",
   "reservation":{"reservation_id":"R-27d48c43","property_id":101,"amount":300,"status":"confirme
d"},
   "notified_subscribers":["notification-service","analytics-service","audit-service"],
   "replicas_updated":["replica-1","replica-2","replica-3"]}}}
```

Pub-sub event and group multicast payloads:

```
{"type":"event","sender":"broker","payload":{"topic":"booking.created","seq":1,
  "publisher":"reservation-rpc","event":{"reservation_id":"R-27d48c43","property_id":101,"amount":30
0}}}

{"type":"group.deliver","sender":"group-coordinator","receiver":"replica-2","payload":{
  "group":"property-replicas","seq":1,"origin":"reservation-rpc",
  "message":{"op":"availability_update","property_id":101,"available":false}}}
```

Message types: `rpc.request`/`rpc.reply`, `subscribe`/`subscribe.ack`, `publish`/`publish.ack`, `event`/`event.ack`, `group.join`/`group.leave`/`group.members`/`group.multicast` (+ `.ack`/`.reply`), `group.deliver`, `group.view`, `group.ack`, `group.heartbeat`, `group.inbox` (hosted).

Second format: typed XML. The same envelope can be sent as XML. Each element carries a type attribute (`t="int|float|bool|str|null|dict|list"`), so the round-trip is lossless:

```
<message><envelope t="dict"><entry key="id"><value t="str">9aee0f075a81</value></entry>
<entry key="type"><value t="str">rpc.request</value></entry> ...
<entry key="payload"><value t="dict"><entry key="params"><value t="dict">
<entry key="property_id"><value t="int">103</value></entry></value></entry></value></entry></envelop
e></message>
```

4. Marshalling and unmarshalling

Step	Path A: Python (`src/communication/codec.py`, `transport.py`)	Path B: Node (`services/shared/messaging.js`) / Worker (`cloudflare/src/comm/marshal.ts`)
Marshal	`marshal(envelope, "json"/"xml")` → UTF-8 bytes (`json.dumps` / ElementTree)	`marshal(envelope, fmt)` → string (`JSON.stringify` / typed-XML serializer)

Step	Path A: Python (`src/communication/codec.py`, `transport.py`)	Path B: Node (`services/shared/messaging.js`) / Worker (`cloudflare/src/comm/marshal.ts`)
Framing	`[len:4][fmt:'J'/'X'][body]`. Removes Milestone 1's 1024-byte `recv` limit and allows many messages per connection	HTTP body. Format given by `Content-Type: application/json` or `application/xml`. The reply uses the same format
Unmarshal	`recv_exact` header → body → `unmarshal(body, fmt)` → dict	`formatFromContentType()` → `unmarshal(text, fmt)` (JSON.parse / small XML parser)
Before/after logging	Each SEND log stores `message` (before) and `wire` (after). Each RECV log stores `wire` (before) and `message` (after)	Identical fields. Hosted: stored in D1 `message_log.message` / `.wire`

5. Why sockets / RPC

	Path A: TCP sockets + JSON-RPC 2.0	Path B: HTTP JSON-RPC 2.0 (+ WebSockets)
Reason	Milestone 1 already used TCP sockets. Sockets show the low-level mechanics the course covers (connect, framing, blocking request-reply). JSON-RPC adds a standard method/params/error contract instead of ad-hoc `action` strings	Cloudflare Workers cannot open listening TCP sockets. Inbound traffic is HTTP/WebSocket only, so RPC over HTTP is the natural fit. JSON-RPC keeps the same method names as Path A (`availability.check`, `reservation.book`, ...). WebSockets give server push for pub-sub and group delivery to browsers or external nodes
Errors	JSON-RPC codes: -32600 invalid request, -32601 method not found, -32602 bad params, 404 property not found	Same codes
Methods	`rpc.ping`, `property.list/get`, `availability.check`, `property.set_available`, `reservation.book/get/list`	Plus `pubsub.publish/subscribers`, `group.join/leave/members/multicast/heartbeat/inbox`, `logs.recent`

RMI was not chosen because it is Java-specific, and the team's stacks are Python and JavaScript/TypeScript.

6. Publish-subscribe design

	Path A: Python	Path B: Node local	Path B: Cloudflare hosted
Broker	`Broker` process :6010	Comm Hub :5010	`CommHub` Durable Object
Subscribe	`subscribe {topics}` on a long-lived socket	`POST /subscribe {topic,name,url}`	Built-in services + `GET /api/comm/ws?role=subscriber&topic=…`
Subscribers in demo	notification-, analytics-, audit-service (3 processes)	same 3 (separate processes)	same 3 built-in + 2 WebSocket clients
Publish	Reservation sends `publish` → broker fans out `event` (with seq) to each subscriber → `publish.ack {delivered_to, failed}`	Same over HTTP. Subscriber replies `event.ack`	Same; WebSocket push
Decoupling / failure	Publisher does not know the subscribers. A broken socket is removed	A subscriber whose push fails is dropped	A closed WebSocket is dropped by the runtime

Topic filtering is tested: a publish on `other.topic` reaches nobody.

7. Group communication design

	Path A: Python	Path B: Node local	Path B: Cloudflare hosted
Coordinator	`GroupCoordinator` :6020	Comm Hub `/group/*`	`CommHub` DO (single instance → one consistent view)

	Path A: Python	Path B: Node local	Path B: Cloudflare hosted
Members	replica-1..3, each with its own TCP listener	replica-1..3 HTTP processes	mailbox members (HTTP join + inbox) and WebSocket members
Membership ops	`group.join/leave/members`; every change → `group.view {view_id, members}` to all	same	same + `group.heartbeat` lease (10 min)
Multicast	Coordinator sends a numbered `group.deliver` to every member and waits for `group.ack`. Replies `{seq, delivered_to, failed}`	same	WebSocket send / mailbox enqueue
Failure detection	Unreachable member is removed and a new view announced	same	Closed socket or expired lease is removed and a new view announced
Use	Keep property availability caches consistent across replicas	same	same

The demos kill `replica-3` (Python, Node) or close the WebSocket replica (Cloudflare). The next multicast reports `failed: ["replica-3"]` and the membership view shrinks.

8. Logging

Every SEND and RECV on every node is logged with an ISO-8601 UTC timestamp (millisecond precision, `Z`):

- **Python:** `logs/messages.jsonl` (`COMM_LOG_FILE` overrides the path), plus readable stdout lines.
- **Node:** `logs/messages.jsonl`, served at `GET /api/comm/logs` (gateway) and `GET /logs` (hub).
- **Cloudflare:** D1 table `message_log`, served at `GET /api/comm/logs?limit=&node=`. It persists across isolates.

9. Log excerpts from real runs

All excerpts were copied from runs on 2026-10-12. The full logs are in `docs/logs/`.

9.1 Path A: Python simulation (`python -m src.communication.demo`)

```
=== 2+3+4. reservation.book -> property RPC, pub-sub, multicast =
2026-10-12T23:16:55.257Z [client] SEND  -> 127.0.0.1:6002 type=rpc.request id=8ef12977323e fmt=json
bytes=266
2026-10-12T23:16:55.266Z [client] RECV  <- 127.0.0.1:6002 type=rpc.reply id=2a0f976ce60b fmt=json by
tes=579
  booking 101: {
  "status": "accepted",
  "reservation": {
    "reservation_id": "R-fa7a220d",
    "property_id": 101,
    "property_name": "Long Beach Apartment",
    "guest_name": "Demo Guest",
    "nights": 2,
    "amount": 300,
    "status": "confirmed",
    "created_at": "2026-10-12T23:16:55.260Z"
...
=== 4b. failure: kill replica-3, multicast again ================
2026-10-12T23:16:55.571Z [client] SEND  -> 127.0.0.1:6020 type=group.multicast id=c308b7a43408 fmt=j
son bytes=270
2026-10-12T23:16:55.576Z [client] RECV  <- 127.0.0.1:6020 type=group.multicast.ack id=906842448018 f
mt=json bytes=282
  multicast ack: {'group': 'property-replicas', 'seq': 2, 'delivered_to': ['replica-1', 'replica-
```

```
2'], 'failed': ['replica-3']}
2026-10-12T23:16:55.576Z [client] SEND  -> 127.0.0.1:6020 type=group.members id=de9e1fae51be fmt=jso
n bytes=194
2026-10-12T23:16:55.577Z [client] RECV  <- 127.0.0.1:6020 type=group.members.reply id=5cb1ab2d7a2f f
mt=json bytes=246
  group members now: ['replica-1', 'replica-2']

=== summary from logs/messages.jsonl ============================
  164 log entries, 73 SEND, 73 RECV
  notification-service   ['send confirmation email to Demo Guest for Long Beach Apartment']
  analytics-service      ['record revenue $300 for property 101']
  audit-service          ['append audit record for R-fa7a220d']
  replica-1              ['replica state now {101: False}', 'replica state now {101: False, 103: Tru
e}']
  replica-2              ['replica state now {101: False}', 'replica state now {101: False, 103: Tru
e}']
```

One SEND entry (booking request) with before/after marshalling:

```
{
 "ts": "2026-10-12T23:16:55.257Z",
 "node": "client",
 "event": "SEND",
 "peer": "127.0.0.1:6002",
 "format": "json",
 "bytes": 266,
 "message": {
  "id": "8ef12977323e",
  "type": "rpc.request",
  "sender": "client",
  "receiver": null,
  "timestamp": "2026-10-12T23:16:55.257Z",
  "correlation_id": null,
  "payload": {
   "jsonrpc": "2.0",
   "method": "reservation.book",
   "params": {
    "property_id": 101,
    "guest_name": "Demo Guest",
    "nights": 2
   },
   "id": 4
  }
 },
 "wire": "{\"id\":\"8ef12977323e\",\"type\":\"rpc.request\",\"sender\":\"client\",\"receiver\":nul
l,\"timestamp\":\"2026-10-12T23:16:55.257Z\",\"correlation_id\":null,\"payload\":{\"jsonrpc\":\"2.0
\",\"method\":\"reservation.book\",\"params\":{\"property_id\":101,\"guest_name\":\"Demo Guest\",\"n
ights\":2},\"id\":4}}"
}
```

9.2 Path B: Node local stack (`npm run comm:demo`)

```
2026-10-12T23:24:19.201Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=aa2856c
aa7ad fmt=json bytes=267
2026-10-12T23:24:19.267Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=6de0db09e
1a6 fmt=json bytes=789
2026-10-12T23:24:19.267Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=b864494
c61c4 fmt=json bytes=241
```

```
2026-10-12T23:24:19.274Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=6d3fa2375
971 fmt=json bytes=267
2026-10-12T23:24:19.277Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=5d4d946
a77f5 fmt=json bytes=324
2026-10-12T23:24:19.286Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=ac97244c4
24e fmt=json bytes=309
2026-10-12T23:24:19.286Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=ec3bf4e
31079 fmt=json bytes=248
```


```
{
 "ts": "2026-10-12T23:24:19.262Z",
 "node": "replica-1",
 "event": "RECV",
 "peer": "127.0.0.1:44818",
 "format": "json",
 "bytes": 334,
 "wire": "{\"id\":\"a6b1d7da05d9\",\"type\":\"group.deliver\",\"sender\":\"comm-hub\",\"receiver
\":\"replica-1\",\"timestamp\":\"2026-10-12T23:24:19.261Z\",\"correlation_id\":null,\"payload\":{\"g
roup\":\"property-replicas\",\"seq\":1,\"origin\":\"reservation-service\",\"message\":{\"op\":\"avai
lability_update\",\"property_id\":101,\"available\":false,\"reservation_id\":\"res-722b39ce\"}}}",
 "message": {
  "id": "a6b1d7da05d9",
  "type": "group.deliver",
  "sender": "comm-hub",
  "receiver": "replica-1",
  "timestamp": "2026-10-12T23:24:19.261Z",
  "correlation_id": null,
  "payload": {
   "group": "property-replicas",
   "seq": 1,
   "origin": "reservation-service",
   "message": {
    "op": "availability_update",
    "property_id": 101,
    "available": false,
    "reservation_id": "res-722b39ce"
   }
  }
 }
}
```


9.3 Path B: Cloudflare Worker (`wrangler dev` + `cloudflare/scripts/comm-demo.mjs`)

```
=== 3. booking -> pub-sub + multicast ============================
  status accepted res-3b8716ac
  notified_subscribers [
  'analytics-service',
  'audit-service',
  'notification-service',
  'ws-subscriber-1-1360',
  'ws-subscriber-2-1360'
]
  replicas_updated [ 'replica-a-1360', 'replica-b-1360', 'replica-ws-1360' ]
  [PASS] booking accepted
  [PASS] built-in + websocket subscribers notified
  [PASS] all 3 replicas got multicast
  [PASS] ws subscriber 1 received event
  [PASS] ws subscriber 2 received event
  [PASS] ws replica received group.deliver
  [PASS] mailbox replica inbox has the update
```

```
  [PASS] double booking rejected

=== 4. failure / leave: websocket replica disconnects ===========
  members now: [ 'replica-a-1360', 'replica-b-1360' ]
  [PASS] closed websocket member removed from view

=== 5. message log (GET /api/comm/logs) ========================
  212 recent entries, 102 from this run
  2026-10-12T23:24:15.352Z [worker-gateway] RECV 127.0.0.1 rpc.request
  2026-10-12T23:24:15.356Z [worker-gateway] SEND 127.0.0.1 rpc.reply
  2026-10-12T23:24:15.363Z [worker-gateway] RECV 127.0.0.1 rpc.request
  2026-10-12T23:24:15.365Z [worker-gateway] SEND 127.0.0.1 rpc.reply
  2026-10-12T23:24:15.375Z [worker-gateway] RECV 127.0.0.1 rpc.request
  2026-10-12T23:24:15.379Z [worker-gateway] SEND 127.0.0.1 rpc.reply
  2026-10-12T23:24:15.385Z [worker-gateway] RECV 127.0.0.1 rpc.request
  2026-10-12T23:24:15.389Z [worker-gateway] SEND 127.0.0.1 rpc.reply
  [PASS] every entry has ISO-8601 ts
  [PASS] log has SEND and RECV for this run

ALL CHECKS PASSED
```

`GET /api/comm/logs` entry (pub-sub fan-out inside the Durable Object):


```
{
 "ts": "2026-10-12T23:23:16.502Z",
 "node": "comm-hub",
 "event": "SEND",
 "peer": "notification-service",
 "format": "json",
 "bytes": 489,
 "note": null,
 "message": {
  "id": "d87f3922dadb",
  "type": "event",
  "sender": "comm-hub",
  "receiver": "notification-service",
  "timestamp": "2026-10-12T23:23:16.502Z",
  "correlation_id": null,
  "payload": {
   "topic": "booking.created",
   "seq": 1,
   "publisher": "worker-gateway",
   "event": {
    "reservation_id": "res-1e2bc68a",
    "property_id": 101,
    "property_name": "Long Beach Apartment",
    "guest_name": "Demo ce07",
    "amount": 150,
    "payment_id": "pay-b670474e",
    "refund_id": null,
    "status": "confirmed",
    "created_at": "2026-10-12T23:23:16.495Z",
    "cancelled_at": null
   }
  }
 },
 "wire": "{\"id\":\"d87f3922dadb\",\"type\":\"event\",\"sender\":\"comm-hub\",\"receiver\":\"notific
ation-service\",\"timestamp\":\"2026-10-12T23:23:16.502Z\",\"correlation_id\":null,\"payload\":{\"to
pic\":\"booking.created\",\"seq\":1,\"publisher\":\"worker-gateway\",\"event\":{\"reservation_id
\":\"res-1e2bc68a\",\"property_id\":101,\"property_name\":\"Long Beach Apartment\",\"guest_name
```

```
\":\"Demo ce07\",\"amount\":150,\"payment_id\":\"pay-b670474e\",\"refund_id\"\u2026"
}
```


Deployment status: the Worker passes every check under `wrangler dev`, but it is **not deployed**. `wrangler deploy` failed 9 times on 2026-10-12 (wrangler 4.135, 4.149 and latest, clean build, with backoff). Cloudflare returned HTTP 500 / code 10013 on `assets-upload-session`, while read calls with the same token work. The live site still serves the Milestone 1 build. To deploy: `npm run cf:deploy` (Node 22+), then `BASE_URL=https://lodging.pitchblack.icu npm run cf:test`.

10. Test cases

#	Test case	Python (`python -m unittest discover -s tests -t . -v`)	Node (`npm test`)	Worker (`npm run cf:test`, BASE_URL)
TC1	JSON + XML marshal round-trip, ISO timestamp	pass	pass	XML reply checked
TC2	Request-reply availability.check / property.get, JSON and XML, error codes	pass	pass	pass
TC3	Booking → 3 subscribers notified + 3 replicas updated; double booking rejected	pass	pass	pass (3 built-in + 2 WS subscribers; 2 mailbox + 1 WS replica)
TC4	Group join/view/multicast; failure removes member; leave	pass	pass	pass
TC5	Pub-sub topic filtering	pass	pass	n/a
TC6	Every log entry ISO-8601; SEND entries have before/after	pass	pass	pass

Results: Python **Ran 10 tests — OK**; Node **7/7 pass**; Worker **all demo checks passed (local wrangler dev)**. The raw outputs are in `docs/logs/*-test-results.txt` and `docs/logs/worker-local-demo-output.txt`.

12. Screenshots

All screenshots come from real runs on 2026-10-12. Terminal output was rendered to PNG, and the UI was captured with headless Chrome. The files are in `docs/milestone-2/screenshots/`.

Figure 01: Python demo, full run (10 processes)

Python simulation: full demo run (10 processes)

```
$ python3 -m src.communication.demo

=== starting nodes (separate processes) ========================
  property               pid=3740938
  broker                 pid=3740939
  coordinator            pid=3740940
  reservation            pid=3740941
  notification-service   pid=3740945
  analytics-service      pid=3740947
  audit-service          pid=3740948
  replica-1              pid=3740949
  replica-2              pid=3740950
  replica-3              pid=3740951

=== marshalling: envelope -> JSON bytes / XML bytes -> envelope =
  json: 231 bytes, round-trip equal = True
    {"id":"4ac3963d6db1","type":"rpc.request","sender":"client","receiver":null,"timestamp":"2026-10-12T23:37:05.431Z","correlation_id":null,"payload":{"jsonrpc":"2...
2026-10-12T23:37:05.431Z [client] INFO  **  type= id= fmt= bytes= marshal round-trip json
  xml: 741 bytes, round-trip equal = True
    <message><envelope t="dict"><entry key="id"><value t="str">4ac3963d6db1</value></entry><entry key="type"><value t="str">rpc.request</value></entry><entry key="s...
2026-10-12T23:37:05.432Z [client] INFO  **  type= id= fmt= bytes= marshal round-trip xml

=== 1. request-reply: availability.check (JSON-RPC over TCP) ====
2026-10-12T23:37:05.432Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=2afe459139db fmt=json bytes=231
2026-10-12T23:37:05.434Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=fbd4f9b42c01 fmt=json bytes=278
  101 -> {'property_id': 101, 'name': 'Long Beach Apartment', 'available': True, 'price': 150}
2026-10-12T23:37:05.434Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=d24ff4e88898 fmt=json bytes=231
2026-10-12T23:37:05.434Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=d395d314d9cc fmt=json bytes=274
  102 -> {'property_id': 102, 'name': 'Downtown Studio', 'available': False, 'price': 120}
2026-10-12T23:37:05.435Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=b4c4ef01ac26 fmt=xml bytes=741
2026-10-12T23:37:05.436Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=6a8fbd25cbba fmt=xml bytes=876
  103 (XML on the wire) -> {'property_id': 103, 'name': 'Beach House', 'available': True, 'price': 275}
2026-10-12T23:37:05.436Z [client] SEND  -> 127.0.0.1:6020 type=group.members id=7fd282aa4eb6 fmt=json bytes=194
2026-10-12T23:37:05.437Z [client] RECV  <- 127.0.0.1:6020 type=group.members.reply id=41c3e7ed9c36 fmt=json bytes=258

  group members: ['replica-1', 'replica-2', 'replica-3']

=== 2+3+4. reservation.book -> property RPC, pub-sub, multicast =
2026-10-12T23:37:05.438Z [client] SEND  -> 127.0.0.1:6002 type=rpc.request id=49cfd1aadd0b fmt=json bytes=266
2026-10-12T23:37:05.447Z [client] RECV  <- 127.0.0.1:6002 type=rpc.reply id=192f939eefe2 fmt=json bytes=579
  booking 101: {
  "status": "accepted",
  "reservation": {
    "reservation_id": "R-9cc0f82a",
    "property_id": 101,
    "property_name": "Long Beach Apartment",
    "guest_name": "Demo Guest",
    "nights": 2,
    "amount": 300,
    "status": "confirmed",
    "created_at": "2026-10-12T23:37:05.441Z"
  },
  "notified_subscribers": [
    "notification-service",
    "audit-service",
    "analytics-service"
  ],
  "replicas_updated": [
    "replica-1",
    "replica-2",
    "replica-3"
  ]
}
2026-10-12T23:37:05.448Z [client] SEND  -> 127.0.0.1:6002 type=rpc.request id=2723f07119d6 fmt=json bytes=257
2026-10-12T23:37:05.449Z [client] RECV  <- 127.0.0.1:6002 type=rpc.reply id=18dba19f87ae fmt=json bytes=280
  booking 101 again: {'status': 'rejected', 'message': 'Property is not available', 'property_id': 101}
2026-10-12T23:37:05.449Z [client] SEND  -> 127.0.0.1:6002 type=rpc.request id=e794464f8911 fmt=json bytes=255
2026-10-12T23:37:05.450Z [client] RECV  <- 127.0.0.1:6002 type=rpc.reply id=26b56462ce43 fmt=json bytes=280
  booking 102: {'status': 'rejected', 'message': 'Property is not available', 'property_id': 102}

=== 4b. failure: kill replica-3, multicast again ===============
2026-10-12T23:37:05.751Z [client] SEND  -> 127.0.0.1:6020 type=group.multicast id=209184c72155 fmt=json bytes=270
2026-10-12T23:37:05.756Z [client] RECV  <- 127.0.0.1:6020 type=group.multicast.ack id=f5f54df71f45 fmt=json bytes=282
  multicast ack: {'group': 'property-replicas', 'seq': 2, 'delivered_to': ['replica-1', 'replica-2'], 'failed': ['replica-3']}
2026-10-12T23:37:05.756Z [client] SEND  -> 127.0.0.1:6020 type=group.members id=acfea9fe659f fmt=json bytes=194
2026-10-12T23:37:05.756Z [client] RECV  <- 127.0.0.1:6020 type=group.members.reply id=fd5549246175 fmt=json bytes=246
  group members now: ['replica-1', 'replica-2']

=== summary from logs/messages.jsonl ===========================
  164 log entries, 73 SEND, 73 RECV
  notification-service   ['send confirmation email to Demo Guest for Long Beach Apartment']
  analytics-service      ['record revenue $300 for property 101']
  audit-service          ['append audit record for R-9cc0f82a']
  replica-1              ['replica state now {101: False}', 'replica state now {101: False, 103: True}']
  replica-2              ['replica state now {101: False}', 'replica state now {101: False, 103: True}']

Full log: /workspace/lodging-main/logs/messages.jsonl
```

Figure 02: Request-reply: JSON-RPC over TCP

Request-reply: JSON-RPC 2.0 over TCP sockets

```
# section 1 of: python3 -m src.communication.demo
=== 1. request-reply: availability.check (JSON-RPC over TCP) ====
2026-10-12T23:37:05.432Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=2afe459139db fmt=json bytes=231
2026-10-12T23:37:05.434Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=fbd4f9b42c01 fmt=json bytes=278
  101 -> {'property_id': 101, 'name': 'Long Beach Apartment', 'available': True, 'price': 150}
2026-10-12T23:37:05.434Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=d24ff4e88898 fmt=json bytes=231
2026-10-12T23:37:05.434Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=d395d314d9cc fmt=json bytes=274
  102 -> {'property_id': 102, 'name': 'Downtown Studio', 'available': False, 'price': 120}
2026-10-12T23:37:05.435Z [client] SEND  -> 127.0.0.1:6001 type=rpc.request id=b4c4ef01ac26 fmt=xml bytes=741
2026-10-12T23:37:05.436Z [client] RECV  <- 127.0.0.1:6001 type=rpc.reply id=6a8fbd25cbba fmt=xml bytes=876
  103 (XML on the wire) -> {'property_id': 103, 'name': 'Beach House', 'available': True, 'price': 275}
2026-10-12T23:37:05.436Z [client] SEND  -> 127.0.0.1:6020 type=group.members id=7fd282aa4eb6 fmt=json bytes=194
2026-10-12T23:37:05.437Z [client] RECV  <- 127.0.0.1:6020 type=group.members.reply id=41c3e7ed9c36 fmt=json bytes=258

  group members: ['replica-1', 'replica-2', 'replica-3']
```

Figure 03: Marshalling before/after (JSON + XML)

Marshalling / unmarshalling: before and after (JSON + XML)

```
# demo marshalling section + entries from logs/messages.jsonl
=== marshalling: envelope -> JSON bytes / XML bytes -> envelope =
  json: 231 bytes, round-trip equal = True
    {"id":"4ac3963d6db1","type":"rpc.request","sender":"client","receiver":null,"timestamp":"2026-10-12T23:37:05.431Z","correlation_id":null,"payload":{"jsonrpc":"2...
2026-10-12T23:37:05.431Z [client] INFO  **  type= id= fmt= bytes= marshal round-trip json
  xml: 741 bytes, round-trip equal = True
    <message><envelope t="dict"><entry key="id"><value t="str">4ac3963d6db1</value></entry><entry key="type"><value t="str">rpc.request</value></entry><entry key="s...
2026-10-12T23:37:05.432Z [client] INFO  **  type= id= fmt= bytes= marshal round-trip xml


--- logs/messages.jsonl: SEND entry (message = BEFORE marshalling, wire = AFTER) ---
{
  "ts": "2026-10-12T23:37:05.432Z",
  "node": "client",
  "event": "SEND",
  "peer": "127.0.0.1:6001",
  "format": "json",
  "bytes": 231,
  "message": {
    "id": "2afe459139db",
    "type": "rpc.request",
    "sender": "client",
    "receiver": null,
    "timestamp": "2026-10-12T23:37:05.432Z",
    "correlation_id": null,
    "payload": {
      "jsonrpc": "2.0",
      "method": "availability.check",
      "params": {
        "property_id": 101
      },
      "id": 1
    }
  },
  "wire": "{\"id\":\"2afe459139db\",\"type\":\"rpc.request\",\"sender\":\"client\",\"receiver\":null,\"timestamp\":\"2026-10-12T23:37:05.432Z\",\"correlation_id\":null...
}

--- RECV entry in XML (wire = BEFORE unmarshalling, message = AFTER) ---
{
  "ts": "2026-10-12T23:37:05.435Z",
  "node": "property-rpc",
  "event": "RECV",
  "peer": "127.0.0.1:40078",
  "format": "xml",
  "bytes": 741,
  "wire": "<message><envelope t=\"dict\"><entry key=\"id\"><value t=\"str\">b4c4ef01ac26</value></entry><entry key=\"type\"><value t=\"str\">rpc.request</value></entry...
  "message": {
    "id": "b4c4ef01ac26",
    "type": "rpc.request",
    "sender": "client",
    "receiver": null,
    "timestamp": "2026-10-12T23:37:05.435Z",
    "correlation_id": null,
    "payload": {
      "jsonrpc": "2.0",
      "method": "availability.check",
      "params": {
        "property_id": 103
      },
      "id": 3
    }
  }
}
```

Figure 04: Pub-sub: booking.created to 3 subscribers

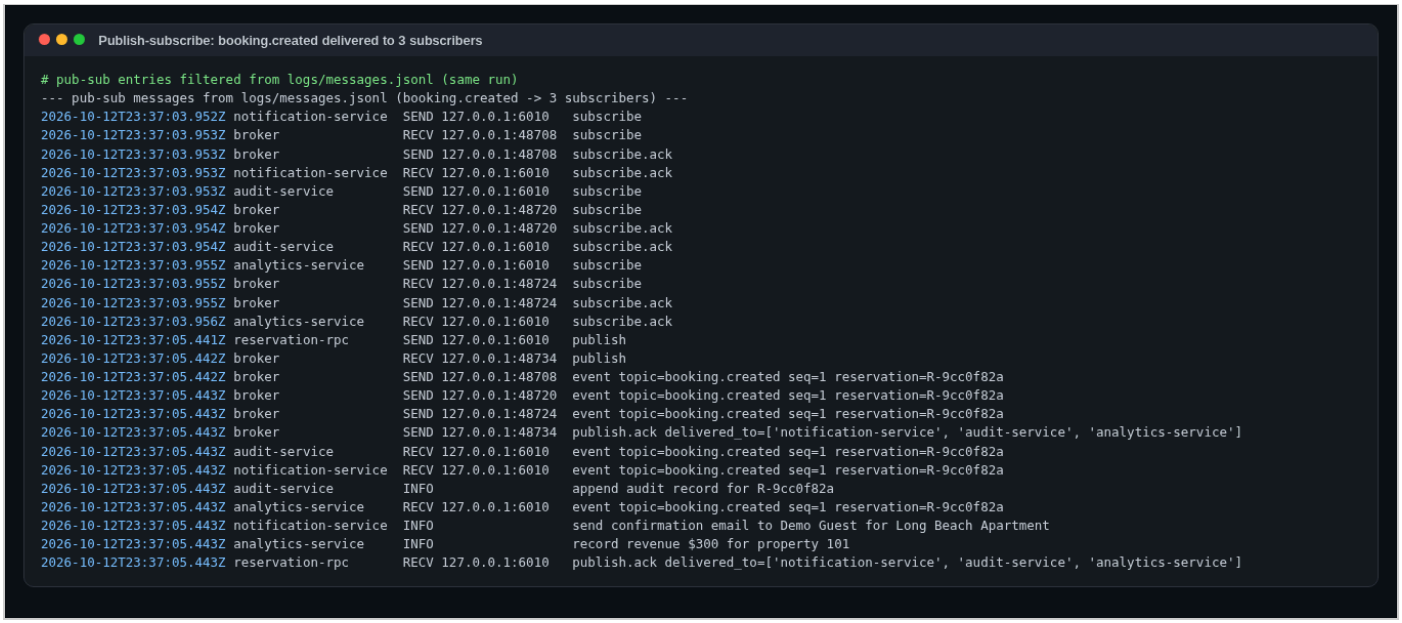


Figure 05: Group communication: multicast, acks, membership, failure

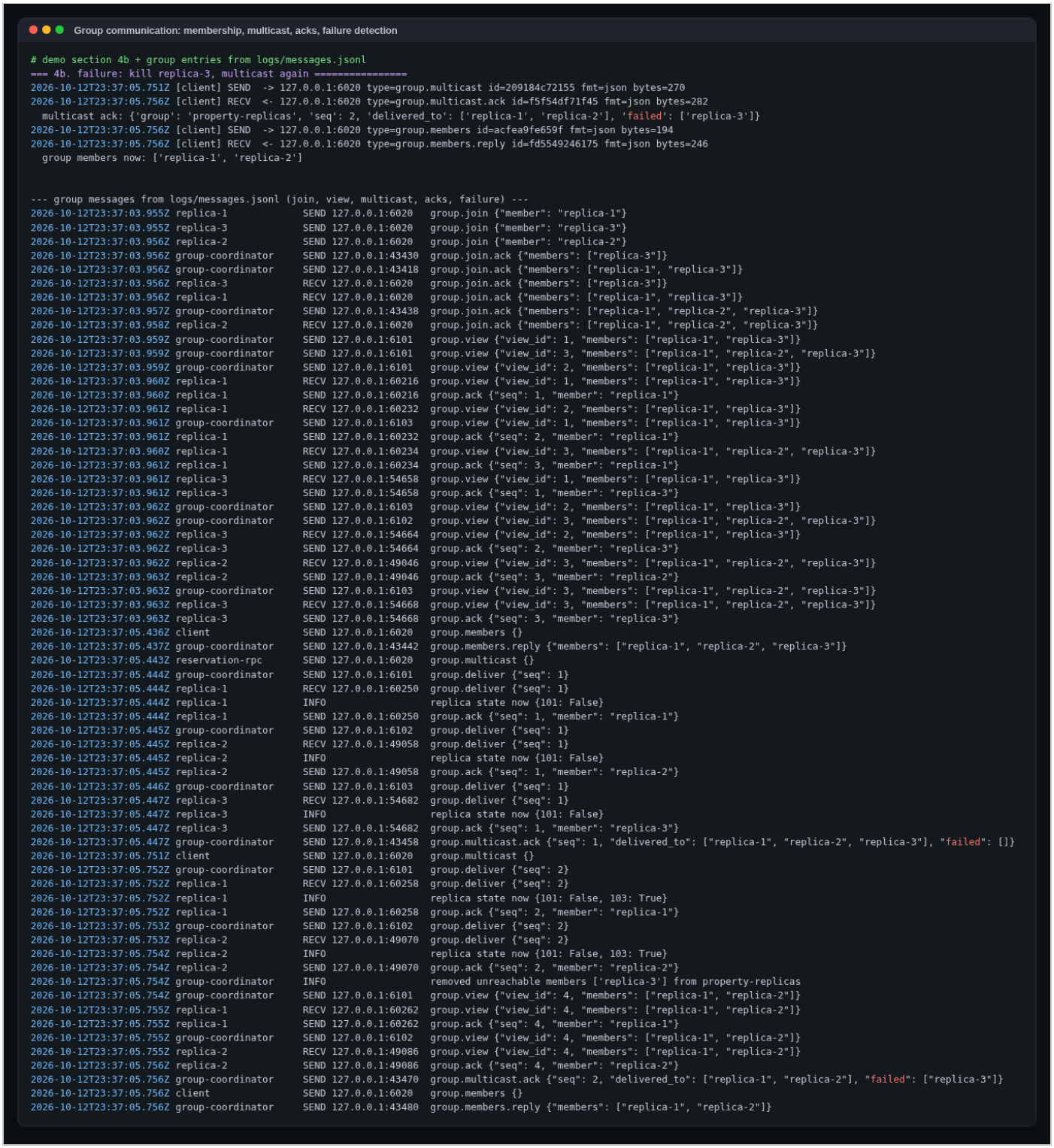


Figure 06: ISO-8601 timestamped messages.jsonl

ISO-8601 timestamped message log

```
$ head -40 logs/messages.jsonl   # long lines cut at 170 chars
{"ts":"2026-10-12T23:37:03.748Z","node":"property-rpc","event":"INFO","note":"listening on 127.0.0.1:6001"}
{"ts":"2026-10-12T23:37:03.749Z","node":"broker","event":"INFO","note":"listening on 127.0.0.1:6010"}
{"ts":"2026-10-12T23:37:03.749Z","node":"group-coordinator","event":"INFO","note":"listening on 127.0.0.1:6020"}
{"ts":"2026-10-12T23:37:03.852Z","node":"reservation-rpc","event":"INFO","note":"listening on 127.0.0.1:6002"}
{"ts":"2026-10-12T23:37:03.952Z","node":"notification-service","event":"SEND","peer":"127.0.0.1:6010","format":"json","bytes":194,"message":{"id":"608904888f12","type"...
{"ts":"2026-10-12T23:37:03.953Z","node":"broker","event":"RECV","peer":"127.0.0.1:48708","format":"json","bytes":194,"wire":"{\"id\":\"608904888f12\",\"type\":\"subscr...
{"ts":"2026-10-12T23:37:03.953Z","node":"broker","event":"SEND","peer":"127.0.0.1:48708","format":"json","bytes":244,"message":{"id":"d2c011d20d46","type":"subscribe.a...
{"ts":"2026-10-12T23:37:03.953Z","node":"notification-service","event":"RECV","peer":"127.0.0.1:6010","format":"json","bytes":244,"wire":"{\"id\":\"d2c011d20d46\",\"ty...
{"ts":"2026-10-12T23:37:03.953Z","node":"audit-service","event":"SEND","peer":"127.0.0.1:6010","format":"json","bytes":187,"message":{"id":"c9814f0e9781","type":"subsc...
{"ts":"2026-10-12T23:37:03.954Z","node":"broker","event":"RECV","peer":"127.0.0.1:48720","format":"json","bytes":187,"wire":"{\"id\":\"c9814f0e9781\",\"type\":\"subscr...
{"ts":"2026-10-12T23:37:03.954Z","node":"replica-1","event":"INFO","note":"listening on 127.0.0.1:6101"}
{"ts":"2026-10-12T23:37:03.954Z","node":"replica-3","event":"INFO","note":"listening on 127.0.0.1:6103"}
{"ts":"2026-10-12T23:37:03.954Z","node":"broker","event":"SEND","peer":"127.0.0.1:48720","format":"json","bytes":230,"message":{"id":"9e9fc7328ff9","type":"subscribe.a...
{"ts":"2026-10-12T23:37:03.954Z","node":"audit-service","event":"RECV","peer":"127.0.0.1:6010","format":"json","bytes":230,"wire":"{\"id\":\"9e9fc7328ff9\",\"type\":\"...
{"ts":"2026-10-12T23:37:03.954Z","node":"replica-2","event":"INFO","note":"listening on 127.0.0.1:6102"}
{"ts":"2026-10-12T23:37:03.955Z","node":"replica-1","event":"SEND","peer":"127.0.0.1:6020","format":"json","bytes":246,"message":{"id":"12d8b74fb9bf","type":"group.joi...
{"ts":"2026-10-12T23:37:03.955Z","node":"replica-3","event":"SEND","peer":"127.0.0.1:6020","format":"json","bytes":246,"message":{"id":"c8668dcd0a87","type":"group.joi...
{"ts":"2026-10-12T23:37:03.955Z","node":"analytics-service","event":"SEND","peer":"127.0.0.1:6010","format":"json","bytes":191,"message":{"id":"7607d3e4827d","type":"s...
{"ts":"2026-10-12T23:37:03.955Z","node":"broker","event":"RECV","peer":"127.0.0.1:48724","format":"json","bytes":191,"wire":"{\"id\":\"7607d3e4827d\",\"type\":\"subscr...
{"ts":"2026-10-12T23:37:03.955Z","node":"broker","event":"SEND","peer":"127.0.0.1:48724","format":"json","bytes":238,"message":{"id":"057f1cb8ff64","type":"subscribe.a...
{"ts":"2026-10-12T23:37:03.955Z","node":"group-coordinator","event":"RECV","peer":"127.0.0.1:43430","format":"json","bytes":246,"wire":"{\"id\":\"c8668dcd0a87\",\"type...
{"ts":"2026-10-12T23:37:03.955Z","node":"group-coordinator","event":"RECV","peer":"127.0.0.1:43418","format":"json","bytes":246,"wire":"{\"id\":\"12d8b74fb9bf\",\"type...
{"ts":"2026-10-12T23:37:03.956Z","node":"replica-2","event":"SEND","peer":"127.0.0.1:6020","format":"json","bytes":246,"message":{"id":"34d2440564a8","type":"group.joi...
{"ts":"2026-10-12T23:37:03.956Z","node":"analytics-service","event":"RECV","peer":"127.0.0.1:6010","format":"json","bytes":238,"wire":"{\"id\":\"057f1cb8ff64\",\"type\...
{"ts":"2026-10-12T23:37:03.956Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:43430","format":"json","bytes":232,"message":{"id":"1620ffa6ad09","type":"...
{"ts":"2026-10-12T23:37:03.956Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:43418","format":"json","bytes":244,"message":{"id":"ac36954711f5","type":"...
{"ts":"2026-10-12T23:37:03.956Z","node":"replica-3","event":"RECV","peer":"127.0.0.1:6020","format":"json","bytes":232,"wire":"{\"id\":\"1620ffa6ad09\",\"type\":\"grou...
{"ts":"2026-10-12T23:37:03.956Z","node":"replica-1","event":"RECV","peer":"127.0.0.1:6020","format":"json","bytes":244,"wire":"{\"id\":\"ac36954711f5\",\"type\":\"grou...
{"ts":"2026-10-12T23:37:03.957Z","node":"group-coordinator","event":"RECV","peer":"127.0.0.1:43438","format":"json","bytes":246,"wire":"{\"id\":\"34d2440564a8\",\"type...
{"ts":"2026-10-12T23:37:03.957Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:43438","format":"json","bytes":256,"message":{"id":"d992121cb9dc","type":"...
{"ts":"2026-10-12T23:37:03.958Z","node":"replica-2","event":"RECV","peer":"127.0.0.1:6020","format":"json","bytes":256,"wire":"{\"id\":\"d992121cb9dc\",\"type\":\"grou...
{"ts":"2026-10-12T23:37:03.959Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:6101","format":"json","bytes":242,"message":{"id":"634e53f9001c","type":"g...
{"ts":"2026-10-12T23:37:03.959Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:6101","format":"json","bytes":254,"message":{"id":"cc3eef66d9de","type":"g...
{"ts":"2026-10-12T23:37:03.959Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:6101","format":"json","bytes":242,"message":{"id":"a1a901f9f3aa","type":"g...
{"ts":"2026-10-12T23:37:03.960Z","node":"replica-1","event":"RECV","peer":"127.0.0.1:60216","format":"json","bytes":242,"wire":"{\"id\":\"634e53f9001c\",\"type\":\"gro...
{"ts":"2026-10-12T23:37:03.960Z","node":"replica-1","event":"SEND","peer":"127.0.0.1:60216","format":"json","bytes":204,"message":{"id":"dbd40a5fc0f2","type":"group.ac...
{"ts":"2026-10-12T23:37:03.960Z","node":"group-coordinator","event":"RECV","peer":"127.0.0.1:6101","format":"json","bytes":204,"wire":"{\"id\":\"dbd40a5fc0f2\",\"type\...
{"ts":"2026-10-12T23:37:03.961Z","node":"replica-1","event":"RECV","peer":"127.0.0.1:60232","format":"json","bytes":242,"wire":"{\"id\":\"a1a901f9f3aa\",\"type\":\"gro...
{"ts":"2026-10-12T23:37:03.961Z","node":"group-coordinator","event":"SEND","peer":"127.0.0.1:6103","format":"json","bytes":242,"message":{"id":"40b687d59210","type":"g...
{"ts":"2026-10-12T23:37:03.961Z","node":"replica-1","event":"SEND","peer":"127.0.0.1:60232","format":"json","bytes":204,"message":{"id":"09c67946df4a","type":"group.ac...
```

Figure 07: Python tests (10/10)

Python tests

```
$ python3 -m unittest discover -s tests -t . -v
test_envelope_has_iso_timestamp (tests.test_communication.TC1_Marshalling.test_envelope_has_iso_timestamp) ... ok
test_json_round_trip (tests.test_communication.TC1_Marshalling.test_json_round_trip) ... ok
test_xml_round_trip_preserves_types (tests.test_communication.TC1_Marshalling.test_xml_round_trip_preserves_types) ... ok
test_availability_check (tests.test_communication.TC2_RequestReply.test_availability_check) ... ok
test_errors (tests.test_communication.TC2_RequestReply.test_errors) ... ok
test_xml_rpc_call (tests.test_communication.TC2_RequestReply.test_xml_rpc_call) ... ok
test_booking_flow (tests.test_communication.TC3_BookingPubSubGroup.test_booking_flow) ... ok
test_join_view_multicast_and_failure (tests.test_communication.TC4_GroupMembership.test_join_view_multicast_and_failure) ... ok
test_only_matching_topic (tests.test_communication.TC5_Pubsub_Topics.test_only_matching_topic) ... Exception in thread Thread-88 (_announce_view):
Traceback (most recent call last):
  File "/usr/lib/python3.13/threading.py", line 1043, in _bootstrap_inner
    self.run()
    ~~~~~~~~^^
  File "/usr/lib/python3.13/threading.py", line 994, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/workspace/lodging-main/src/communication/group.py", line 71, in _announce_view
    self._fanout(group, "group.view", view)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/workspace/lodging-main/src/communication/group.py", line 59, in _fanout
    (delivered if self._deliver(member, addr, env) else failed).append(member)
                  ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^
  File "/workspace/lodging-main/src/communication/group.py", line 48, in _deliver
    ack, _ = recv_message(s, self.logger, peer)
             ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^
  File "/workspace/lodging-main/src/communication/transport.py", line 47, in recv_message
    (length,) = HEADER.unpack(_recv_exact(sock, HEADER.size))
                              ~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
  File "/workspace/lodging-main/src/communication/transport.py", line 34, in _recv_exact
    raise ConnectionClosed("peer closed the connection")
src.communication.transport.ConnectionClosed: peer closed the connection
ok
test_every_entry_is_timestamped (tests.test_communication.TC6_Logging.test_every_entry_is_timestamped) ... ok

----------------------------------------------------------------------
Ran 10 tests in 0.705s

OK
```

Figure 08: Node comm demo

Node stack: communication demo (11 processes)

```
$ npm run comm:demo

=== processes ================================================
  property               pid=3741156
  payment                pid=3741157
  hub                    pid=3741158
  reservation            pid=3741183
  gateway                pid=3741184
  notification-service   pid=3741197
  analytics-service      pid=3741198
  audit-service          pid=3741199
  replica-1              pid=3741205
  replica-2              pid=3741211
  replica-3              pid=3741217

=== marshalling round-trip ===================================
  json: 231 bytes, round-trip equal = true
    {"id":"cf80e08714e6","type":"rpc.request","sender":"client","receiver":null,"timestamp":"2026-10-12T23:37:07.441Z","correlation_id":null,"payload":{"jsonrpc":"2...
2026-10-12T23:37:07.442Z [client] INFO  **  type= id= fmt= bytes= marshal round-trip json
  xml: 739 bytes, round-trip equal = true
    <message><envelope t="dict"><entry key="id"><value t="str">cf80e08714e6</value></entry><entry key="type"><value t="str">rpc.request</value></entry><entry key="s...
2026-10-12T23:37:07.442Z [client] INFO  **  type= id= fmt= bytes= marshal round-trip xml

=== 1. request-reply: JSON-RPC availability.check ===============
2026-10-12T23:37:07.443Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=590508b4a4c8 fmt=json bytes=243
2026-10-12T23:37:07.478Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=7898fec07525 fmt=json bytes=286
  101 -> {
    "property_id": 101,
    "name": "Long Beach Apartment",
    "available": true,
    "price": 150
  }
2026-10-12T23:37:07.478Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=41a18b7be98e fmt=json bytes=243
2026-10-12T23:37:07.485Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=426e4109704b fmt=json bytes=282
  102 -> {
    "property_id": 102,
    "name": "Downtown Studio",
    "available": false,
    "price": 120
  }
2026-10-12T23:37:07.485Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=4c9027182b1a fmt=xml bytes=751
2026-10-12T23:37:07.493Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=9f556dbea4fa fmt=xml bytes=884
  103 (XML on the wire) -> {
    "property_id": 103,
    "name": "Beach House",
    "available": true,
    "price": 275
  }
2026-10-12T23:37:07.493Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=1d87675eec72 fmt=json bytes=243
2026-10-12T23:37:07.499Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=27a59c1e3c5d fmt=json bytes=277
  999 -> {
    "code": 404,
    "message": "Property not found",
    "data": {
      "property_id": 999
    }
  }
2026-10-12T23:37:07.499Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=8c450ec7eae6 fmt=json bytes=248
2026-10-12T23:37:07.503Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=3bf1a84290ea fmt=json bytes=285
  group members: [
    "replica-1",
    "replica-2",
    "replica-3"
  ]

=== 2+3+4. reservation.book -> payment, pub-sub, multicast ======
2026-10-12T23:37:07.503Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=9797768878d2 fmt=json bytes=267
2026-10-12T23:37:07.570Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=065daa3f448b fmt=json bytes=789
  booking 101: {
    "status": "accepted",
    "reservation_id": "res-c5343878",
    "amount": 150,
    "notified_subscribers": [
      "analytics-service",
      "audit-service",
      "notification-service"
    ],
    "replicas_updated": [
      "replica-1",
      "replica-2",
      "replica-3"
    ]
  }
2026-10-12T23:37:07.570Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=51c5eeddc26a fmt=json bytes=241
2026-10-12T23:37:07.577Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=d915a68bd33d fmt=json bytes=267
  booking 101 again: {
    "status": "rejected",
    "message": "Property is not available"
  }

=== 4b. failure: kill replica-3, multicast again ================
2026-10-12T23:37:07.581Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=2f69203179ec fmt=json bytes=324
2026-10-12T23:37:07.590Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=42d41e61c794 fmt=json bytes=309
  multicast ack: {
    "group": "property-replicas",
    "seq": 2,
    "delivered_to": [
      "replica-1",
      "replica-2"
    ],
    "failed": [
      "replica-3"
```

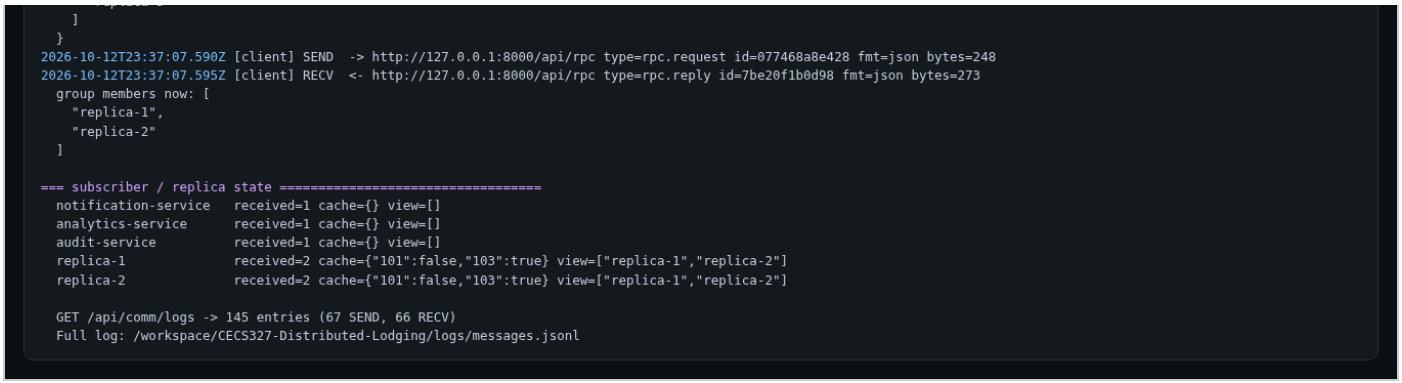


Figure 09: Node tests (7/7)

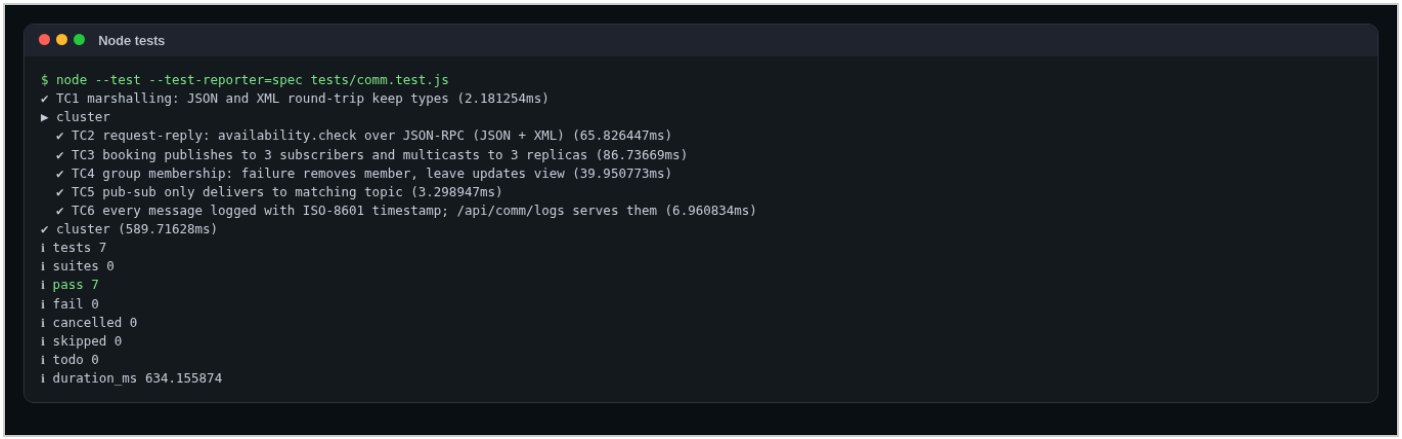


Figure 10a: Worker under wrangler dev: 20/20 checks

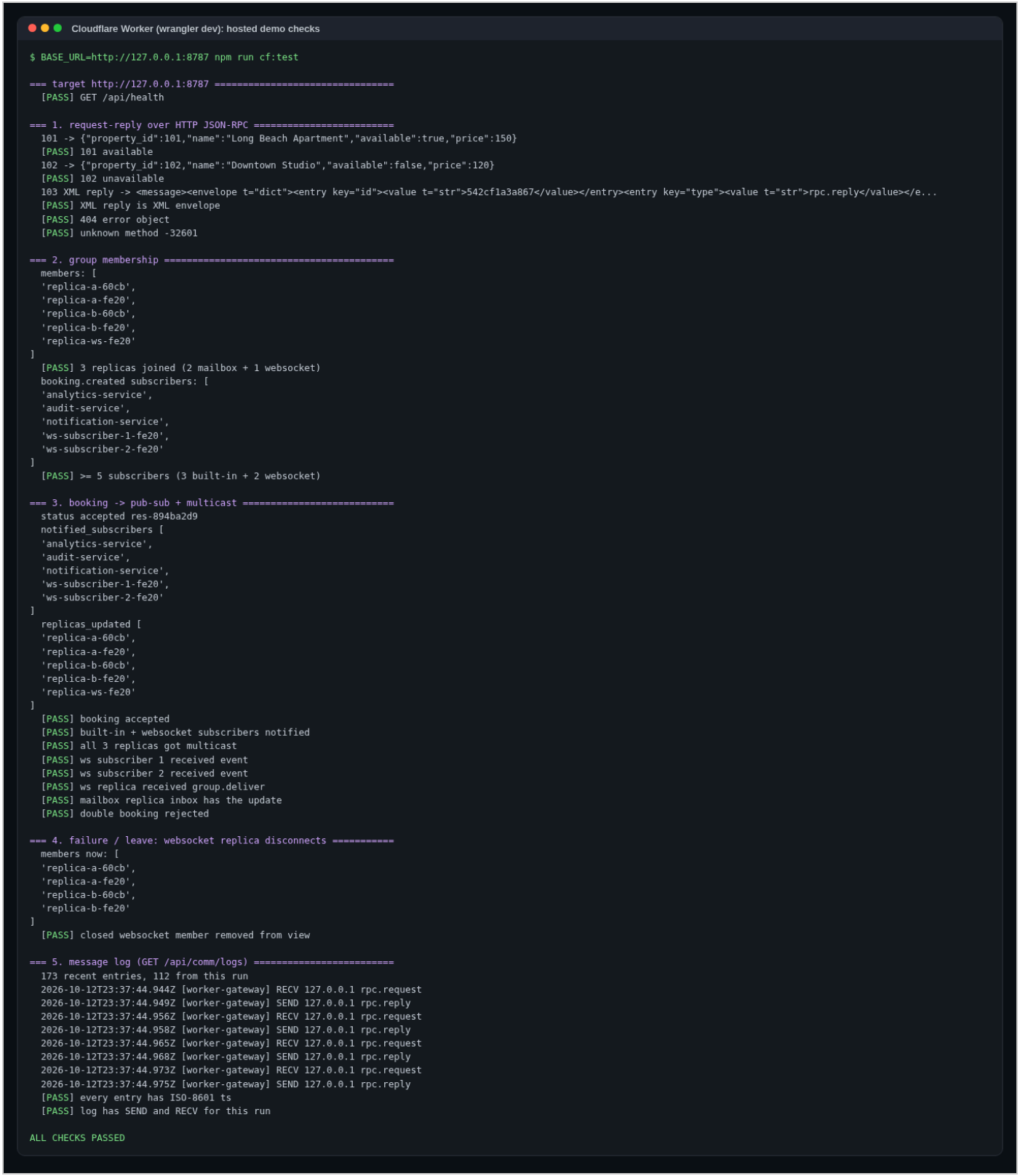
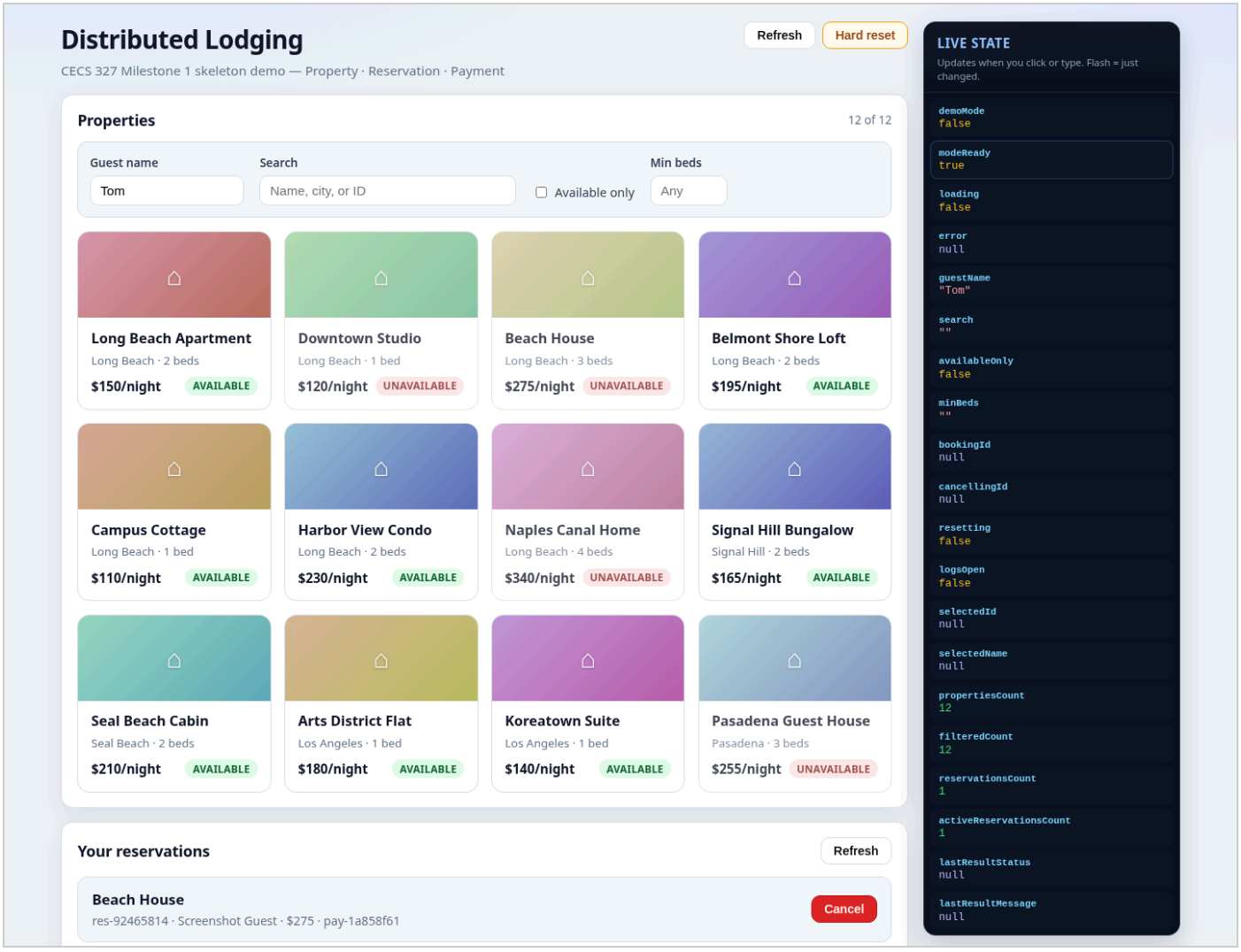


Figure 10b: Worker GET /api/comm/logs JSON

Cloudflare Worker: GET /api/comm/logs (D1 message_log)

```
# curl -s 'http://127.0.0.1:8787/api/comm/logs?limit=10&node=comm-hub'  (last 6 entries; wire/message trimmed)
{
  "status": "ok",
  "logs": [
    {
      "ts": "2026-10-12T23:37:47.367Z",
      "node": "comm-hub",
      "event": "RECV",
      "peer": "demo-client-fe20",
      "format": "json",
      "bytes": 219,
      "note": null,
      "message": {
        "id": "2a3965d6193b",
        "type": "group.leave",
        "sender": "demo-client-fe20",
        "receiver": "comm-hub",
        "timestamp": "2026-10-12T23:37:47.366Z",
        "correlation_id": null
      },
      "wire": "{\"id\":\"2a3965d6193b\",\"type\":\"group.leave\",\"sender\":\"demo-client-fe20\",\"receiver\":\"comm-hub\",\"timestamp\":\"2026-10-12T23:3..."
    },
    {
      "ts": "2026-10-12T23:37:47.367Z",
      "node": "comm-hub",
      "event": "SEND",
      "peer": "mailbox:replica-a-60cb",
      "format": "json",
      "bytes": 249,
      "note": null,
      "message": {
        "id": "1420f4400e20",
        "type": "group.view",
        "sender": "comm-hub",
        "receiver": "replica-a-60cb",
        "timestamp": "2026-10-12T23:37:47.367Z",
        "correlation_id": null
      },
      "wire": "{\"id\":\"1420f4400e20\",\"type\":\"group.view\",\"sender\":\"comm-hub\",\"receiver\":\"replica-a-60cb\",\"timestamp\":\"2026-10-12T23:37:4..."
    },
    {
      "ts": "2026-10-12T23:37:47.367Z",
      "node": "comm-hub",
      "event": "SEND",
      "peer": "mailbox:replica-b-60cb",
      "format": "json",
      "bytes": 249,
      "note": null,
      "message": {
        "id": "60ea3bbec043",
        "type": "group.view",
        "sender": "comm-hub",
        "receiver": "replica-b-60cb",
        "timestamp": "2026-10-12T23:37:47.367Z",
        "correlation_id": null
      },
      "wire": "{\"id\":\"60ea3bbec043\",\"type\":\"group.view\",\"sender\":\"comm-hub\",\"receiver\":\"replica-b-60cb\",\"timestamp\":\"2026-10-12T23:37:4..."
    },
    {
      "ts": "2026-10-12T23:37:47.367Z",
      "node": "comm-hub",
      "event": "SEND",
      "peer": "demo-client-fe20",
      "format": "json",
      "bytes": 253,
      "note": null,
      "message": {
        "id": "80443d33a195",
        "type": "group.leave.ack",
        "sender": "comm-hub",
        "receiver": "demo-client-fe20",
        "timestamp": "2026-10-12T23:37:47.367Z",
        "correlation_id": "2a3965d6193b"
      },
      "wire": "{\"id\":\"80443d33a195\",\"type\":\"group.leave.ack\",\"sender\":\"comm-hub\",\"receiver\":\"demo-client-fe20\",\"timestamp\":\"2026-10-12T..."
    },
    {
      "ts": "2026-10-12T23:37:47.374Z",
      "node": "comm-hub",
      "event": "INFO",
      "peer": null,
      "format": null,
      "bytes": null,
      "note": "websocket subscriber ws-subscriber-1-fe20 closed"
    },
    {
      "ts": "2026-10-12T23:37:47.374Z",
      "node": "comm-hub",
      "event": "INFO",
      "peer": null,
      "format": null,
      "bytes": null,
      "note": "websocket subscriber ws-subscriber-2-fe20 closed"
    }
  ]
}
```

Figure 11: React UI (local stack) after a booking made through /api/rpc



13. Contribution table

Member	Contribution (communication milestone)
Member 1	*TBD*
Member 2	*TBD*
Member 3	*TBD*
Member 4	*TBD*
Member 5	*TBD*
Member 6	*TBD*
Member 7	*TBD*

Fill in names and contributions before submitting.